
Supervised and Self-Supervised Learning on Food-251

Anonymous Author(s)

Affiliation, Address

anon.email@example.org

Abstract

1 We train a residual convolutional network from scratch to classify the 251 fine-
2 grained food categories of FoodX-251, under a hard budget of ten million trainable
3 parameters and a single 8GB laptop GPU. Short 15-epoch proxy runs on a held-
4 out development split screen trunk variants and recipe choices cheaply; one full-
5 length run on the selected configuration then reaches **63.83% top-1 / 87.39%**
6 **top-5** on a test split read exactly once, using 66% of the parameter budget. Two
7 results shape that outcome. Capacity never binds: a trunk 21% smaller ties the
8 adopted one exactly (McNemar $p = 1.0$), and the accuracy cliff lies somewhere
9 below that, bracketed between 1.68M and 5.18M parameters. Nor does overfitting:
10 six regularization axes were each proxied and none beat the plain recipe, because
11 the model underfits at every schedule length tested – the validation-over-training
12 accuracy gap closes from +11.1 points at 15 epochs to zero only at 90. Throughout,
13 measured wall-clock and memory behavior on commodity hardware shapes the
14 design as much as parameter or FLOP counts do. A comparison against self-superv-
15 ised pretraining under an equal-GPU-hour budget is designed and implemented
16 but not yet run, and is reported as such.

17 1 Introduction

18 FoodX-251 is a fine-grained food image classification benchmark spanning 251 categories, built from
19 a large, web-crawled, noisily labeled training set with a smaller, manually curated and clean valida-
20 tion set [1]. Fine-grained food categories are hard to separate because inter-class visual differences
21 (plating, preparation, garnish) can be subtler than the intra-class variation induced by lighting, angle,
22 and portioning, and the label noise inherent to a web-crawled training set compounds the problem.

23 Two constraints are imposed by the setting rather than chosen for effect: the classifier trains from
24 scratch, with no ImageNet or other pretraining, under fewer than 10,000,000 trainable parameters,
25 and every run happens on a single consumer laptop GPU with 8GB of VRAM. Together they shape
26 everything that follows – most of all by making wall-clock, not parameters, the resource actually in
27 short supply.

28 We make three contributions, in the order the underlying work was carried out. First, a budget-
29 constrained architecture search (Section 5.1) that screens residual-trunk variants on measured
30 throughput and memory before accuracy, and that probes **below** the adopted trunk as well as above
31 it – finding that the parameter cap never binds, and bracketing the width at which accuracy does
32 start to fall. Second, a six-axis recipe search (Section 5.2) in which every regularizer tried lost to
33 the plain recipe, with a paired significance test separating the losses that are real from the one that
34 is merely unproven, and a single diagnosis – underfitting – that accounts for all six. Third, a full-
35 length run confirming the resulting configuration on a test split touched exactly once (Section 5.3).
36 A fourth strand, a comparison against self-supervised pretraining under an equal-GPU-hour budget
37 (Section 5.4), is designed and implemented but unrun; it is presented as a design, without numbers.

38 One design choice is worth flagging before the rest. Despite FoodX-251’s long-tailed reputation, the
39 training split we measured is close to class-balanced (median 471 images per class; a single 34-
40 image class drives the oft-cited 19.3x imbalance ratio), so class-balanced losses, logit adjustment,
41 and long-tail-specific training are out of scope here – the real distributional problem is label noise,
42 not long-tailedness (Section 3).

43 2 Related work

44 Three lanes matter here, and all three share one gap.

45 **Parameter-efficient architecture.** MobileNetV4 [2] and MobileOne [3] optimize for on-device
46 latency rather than parameter count alone – the same distinction Section 5.1 arrives at by measure-
47 ment – while ConvNeXt V2 [4] revisits convolutional design at a scale where masked-autoencoder
48 pretraining is available, a regime the from-scratch constraint excludes.

49 **Augmentation and label-noise robustness.** RandAugment [5] and TrivialAugment [6] establish
50 that automated or tuning-free policies match searched ones at near-zero search cost; mixup [7] and
51 CutMix [8] mix inputs and labels instead of choosing among fixed operators; and Generalized Cross
52 Entropy [9] offers a noise-robust loss, directly relevant to web-crawled training labels. Section 5.2
53 tests all five.

54 **Self-supervised pretraining.** SimCLR [10] needs a large negative-sample batch, unreachable at
55 8GB. BYOL [11] and SimSiam [12] avoid negative pairs entirely and stay viable at small batch sizes,
56 which is what makes them the basis of the design in Section 5.4.

57 **The gap.** We found no source evaluating any of this on FoodX-251, Food-101 or Food-2K under
58 a hard parameter budget, and none ablating sub-10M-parameter models trained for tens of epochs.
59 Whether the accepted wisdom transfers to that regime is an open question here, which is why
60 Section 5.2 re-tests techniques well established elsewhere – and why several do not survive.

61 3 Dataset and protocol

62 FoodX-251 [1] contains 251 fine-grained food categories: a large, web-crawled training set with noisy
63 labels, and a smaller, manually curated validation set of 11,994 images.

64 **Class balance.** Despite FoodX-251’s long-tailed reputation, per-class training counts are close to
65 balanced: median 471 images per class, 10th/90th percentile 366/580, and exactly one class below
66 200 images (34) – that single outlier drives the frequently cited 19.3x imbalance ratio. We therefore
67 do not use class-balanced losses, logit adjustment, or decoupled classifier retraining; the real distrib-
68 utional challenge is label noise, since training labels are web-crawled and validation labels are clean,
69 not long-tailedness.

70 **Validation split.** Ablations and checkpoint selection must never touch the number this report
71 ultimately quotes, or that number is implicitly selected-on. We split the 11,994-image validation set
72 50/50, stratified per class (every one of the 251 classes lands within one image of an exact half), into
73 val-dev (6,063 images) and val-test (5,931 images), deterministically: `src/make_val_split.py`
74 is a pure function of the official validation labels and a fixed seed (251), so the split is byte-identical
75 on every regeneration and is not committed (`splits/val_split.csv` is gitignored and regenerated
76 once per checkout). Every ablation and architecture comparison in this report reads val-dev; val-
77 test is read exactly once, for the headline number in Section 5.3.

78 **Proxy protocol and logging.** At about 1.2 minutes/epoch a 15-epoch run costs roughly 18 minutes,
79 making comparisons that would be unaffordable at full length routine. Every comparison below ranks
80 candidates on a 15-epoch val-dev proxy, with the winner confirmed by one full-length run; every
81 run is logged with a config hash, per-epoch train/validation accuracies, wall-clock and peak VRAM
82 (`src/runlog.py`), so each number here traces back to a specific run.

83 **Seeding.** No run in this project was seeded, so every accuracy figure below is a point estimate from
84 one unrepeatable run rather than a mean over seeds. This bounds what the comparisons can claim:
85 the paired McNemar tests used throughout account for **sampling** variance over images and are silent

86 on the training variance reseeding would expose. Two comparisons here are close enough for that to
87 matter; both are flagged where they occur.

88 4 Method

89 4.1 Architecture

90 FoodCNN (`src/model.py`) is a plain residual CNN, sized to spend its parameter budget where it
91 can affect accuracy rather than where convention places it. A stem (3x3 stride-2 convolution, batch
92 normalization, ReLU, 3x3 stride-2 max-pool) takes the input to a quarter of its input resolution before
93 any residual stage runs. Four residual stages follow, each halving spatial resolution and doubling
94 channel width relative to the last – 64, 128, 256, then 512 channels, with block counts $[2, 2, 2, 1]$ –
95 so that the final stage, carrying one block rather than two, is what keeps the trunk under budget.

96 Each residual block normalizes after both convolutions and, whenever the shortcut is not the identity,
97 after the 1x1 shortcut projection too – an unnormalized branch should not be added to a normalized
98 one. The second batch norm’s scale (γ) is zero-initialized, so every block begins as an identity map,
99 which is what makes a high learning rate safe under warmup; and the nonlinearity is applied after
100 the residual addition, so each block is an actual nonlinearity rather than an affine detour.

101 **A genuinely global head.** Pooling to $(1, 1)$ rather than the $(7, 7)$ map convolutions naturally leave
102 behind matters: at $(7, 7)$, flattening produces a $512 \times 7 \times 7 = 25,088$ -feature vector, and a single
103 subsequent linear layer to 251 classes costs 6.4M parameters – 94% of the entire model, spent on
104 its least expressive layer. Pooling to $(1, 1)$ first reduces that same layer to `Linear(512, 251)`, 129k
105 parameters.

106 **Parameter budget.** With the fixes above, FoodCNN totals 6,576,955 trainable parameters – 65.8%
107 of the 10M budget – with roughly 98% of them inside the convolutional trunk rather than the
108 classifier head.

109 **A rejected alternative.** A MobileNetV2-style depthwise-separable version of the same trunk
110 measured 2.55M parameters, 621 img/s and 3.83 GiB peak against the plain trunk’s 6.58M, 918 img/
111 s and 1.28 GiB (Section E). Fewer parameters, slower and heavier: depthwise convolutions trade
112 parameter count for activation memory and wall-clock, which is the wrong trade when throughput
113 and memory are what bind (Section 6).

114 4.2 Training recipe

115 The reference training loop is modernized in ways that cost nothing at this parameter budget: mixed-
116 precision `torch.autocast`, with the dtype chosen from the GPU’s compute capability rather than
117 hardcoded – bf16 without a `GradScaler` on Ampere and later (bf16 needs no dynamic loss scaling),
118 fp16 with one below it, since bf16 has no Tensor Core path on older cards and falls back to an
119 emulation roughly an order of magnitude slower – `channels_last` memory format on both model
120 and inputs, a cosine learning-rate schedule with five-epoch linear warmup in place of step decay,
121 `CrossEntropyLoss` with label smoothing 0.1, SGD with Nesterov momentum, eight dataloader
122 workers (up from four), a 176px training crop (down from 224), and no `DataParallel` wrapper –
123 there is only one GPU to shard across, and the wrapper’s scatter/gather adds a per-step cost and a
124 `module.prefix` to every checkpoint key for no benefit here.

125 4.3 Recipe extensions

126 Four further axes are opt-in flags, all defaulting to off: an augmentation switch between none,
127 `TrivialAugment` [6] and `RandAugment` [5]; batch-wise Mixup [7] and CutMix [8], scored against the
128 pre-mix hard label since a mixed batch has no single correct class; an exponential moving average of
129 weights (`AveragedModel`, batch-norm statistics left live rather than averaged); and Generalized Cross
130 Entropy [9] in place of label-smoothed cross-entropy, for the training set’s label noise.

131 5 Experiments

Accuracy figures below are val-dev unless stated otherwise; val-test is read exactly once, in Section 5.3. All measurements come from the project GPU (RTX 5050 Laptop, 8GB VRAM) in its performance power profile, a qualifier Section 6 shows matters for every throughput number here.

5.1 Architecture search

A first pass screened trunk variants on synthetic throughput and memory alone (Section D), before spending GPU-hours on accuracy. Two findings from it decided which variants earned an accuracy proxy: stage widths must be multiples of 32, since a 72-144-288-576 trunk has **fewer** parameters than the five-stage candidate yet runs 46% slower off the tensor-core fast path; and final spatial resolution dominates every other knob, with the stem max-pool’s removal costing 2.9x throughput and 2x memory at identical parameter count. The four survivors were then measured for accuracy over 15-epoch val-dev proxies (benchmarks/proxy_sweep.py):

Table 1: 15-epoch val-dev proxy results for four candidate trunks, all under the 10M-parameter cap.

Variant	Params	val-dev top-1	val-dev top-5	Peak VRAM
[2,2,2,2] 64-448	9.46M	49.99%	78.29%	2.04 GiB
[2,2,4,1] 64-512	8.94M	49.73%	77.93%	2.14 GiB
baseline [2,2,2,1] 64-512	6.58M	48.82%	77.12%	1.99 GiB
5-stage 48-512 [2,2,2,2,1]	9.08M	47.42%	75.94%	1.61 GiB

The five-stage variant’s extra downsample to a 3x3 final map costs 1.4-2.6 accuracy points despite being the second-largest candidate by parameter count, so the throughput and memory advantage it showed in the synthetic pass does not transfer to a real training signal; it is ruled out. The remaining three sit within 1.2 points of each other on one unseeded run each – not enough to call a winner with confidence. We adopt the baseline [2,2,2,1] 64-512 trunk provisionally: it is statistically indistinguishable from the other two here, uses 30% fewer parameters than the nominal leader, and that headroom has not measured as worth anything yet – better spent, for now, on the recipe tuning in Section 5.2. [2,2,4,1] and [2,2,2,2] 64-448 remain defined in benchmarks/trunk_variants.VARIANTS and can be re-proxied without repeating the measurement work above, if the tuned baseline underperforms expectations.

Below the baseline, and the head. That sweep spans 6.58M to 9.46M – the baseline and upward – but diminishing returns going up say nothing about going down, and the pooling head had never been an experimental axis at all. Five further 15-epoch legs close both gaps. They ran under the recipe Section 5.2 settles on (plain, lr 0.8) rather than the lr 0.1 above, so they are not comparable to Table 1, which is why the baseline was re-run as its own leg. Each is compared against it by a paired McNemar exact test on the same 6,063 images (benchmarks/significance_test.py), since a point-estimate gap alone cannot say whether a comparison had the power to detect a difference:

Table 2: Trunk widths below the baseline, and two alternative pooling heads. 15-epoch val-dev proxies, plain recipe, lr 0.8, batch 256; p from a paired McNemar exact test against the baseline leg.

Variant	Params	top-1	Δ	McNemar p	Peak VRAM
narrow [2,2,2,1] 64-384	5.18M	55.22%	+0.00	1.0000	1.96 GiB
baseline [2,2,2,1] 64-512	6.58M	55.22%	—	—	1.99 GiB
head: spatial attention	6.59M	55.01%	−0.21	0.6826	1.99 GiB
narrow [2,2,2,1] 32-256	1.68M	48.64%	−6.58	< 0.0001	1.08 GiB
head: GAP+GMP concat	6.71M	37.51%	−17.71	< 0.0001	1.99 GiB

The parameter cap does not bind, and now there is evidence on both sides of the baseline rather than one. At 21% fewer parameters, 64-384 is not merely close to the baseline but a literal dead heat: 441 images the baseline alone got right, 441 the narrow trunk alone got right, $p = 1.0000$. Halving again to 32-256 then costs 6.58 points at $p < 0.0001$. So the plateau extends below the adopted trunk, and it ends in a cliff rather than a slope, somewhere between 1.68M and 5.18M parameters. We keep

the 6.58M baseline anyway, since 64-384 buys no accuracy, no meaningful wall-clock (20 vs. 21 minutes) and no VRAM that matters, while the baseline is the trunk every recipe measurement below was taken on. On a task graded against a parameter cap, it is worth stating plainly that the cap was never the difficulty.

Neither pooling head displaces global average pooling. Spatial attention is a clean null (-0.21 , $p = 0.68$): learned pooling weights bought nothing over uniform averaging. The GAP+GMP concatenation’s collapse is a different failure – its train loss starts **above** the baseline’s at epoch 0 (5.595 vs. 4.996) and never closes, so it underfits rather than diverges. Max-pooled activations are unbounded and much larger than mean-pooled ones, so concatenating hands the classifier two feature blocks on very different scales, against a learning rate tuned for a head without that problem. It needs its own learning rate or a normalization layer; that is not the same as being wrong for this dataset.

5.2 Recipe ablation

Six axes were swept on the selected trunk with 15-epoch val-dev proxies: learning rate, augmentation policy, Mixup/CutMix, weight averaging, training loss, and batch size, plus the RandomResizedCrop scale floor. **Every one lost to the plain recipe.** This section separates the losses a significance test calls real from the one it does not, then gives the diagnosis behind all six.

Learning rate, the one axis that paid. Sweeping lr at batch 256 gives 43.96%, 49.41%, 52.56%, 54.02%, 54.96%, 55.58% top-1 for lr 0.05, 0.1, 0.2, 0.4, 0.6, 0.8 (per-epoch curves in Section B). The gain per doubling falls cleanly – +5.45, +3.15, +1.46, +0.94, +0.62 – so a further doubling to 1.6 would be worth perhaps 0.3-0.4 points. Against that, lr 0.8’s validation curve is visibly noisier mid-run than 0.4’s or 0.6’s, though its training loss never wobbles and reaches the lowest final value of the sweep: a noisier optimization regime, not an unstable one. Shrinking reward against growing noise is where we stop. **lr 0.8 is what the rest of this section, and Section 5.3, carry forward** – an 8x increase over the reference script’s default, and worth 5.6 points over the lr 0.1 it replaces.

Everything meant to regularize. The remaining axes are compared against a plain-recipe control at the same lr, each with a paired McNemar exact test on the same 6,063 val-dev images. Since top-1 alone could hide a rare-class or calibration trade-off, benchmarks/analyze_errors.py scores every checkpoint on macro-F1, the mean accuracy of the 30 worst classes, and 15-bin expected calibration error:

Table 3: Five regularization axes against the plain-recipe control. 15-epoch val-dev proxies, lr 0.8, batch 256. “worst-30” is the mean accuracy of the 30 lowest-accuracy classes; ECE is 15-bin expected calibration error.

Recipe	top-1	macro-F1	worst-30	ECE	McNemar p
plain (control)	55.58%	53.38%	16.77%	0.182	—
TrivialAugment	52.09%	49.63%	14.05%	0.180	< 0.0001
RandAugment	52.60%	50.22%	14.78%	0.184	< 0.0001
CutMix	48.84%	45.61%	7.19%	0.172	< 0.0001
Mixup	42.95%	40.05%	5.13%	0.231	< 0.0001
EMA (decay 0.999)	39.88%	38.32%	4.52%	0.107	< 0.0001
GCE loss ($q = 0.7$)	22.23%	14.86%	0.00%	0.328	< 0.0001

The control leads on top-1, macro-F1 and worst-30 simultaneously, so none of these is quietly buying rare-class recall at the cost of overall accuracy. Two of the losses deserve a caveat rather than a verdict. EMA’s averaging window at decay 0.999 is roughly 1,000 steps against a proxy only 6,945 steps long, so its weights may still be lagging the raw, rapidly-moving ones when the proxy ends – note that it is the best-calibrated run in the table, which is what a lagging average would look like. GCE’s collapse is more extreme than a merely bad hyperparameter usually produces – 119 of 251 classes at zero accuracy – but the shape of the failure is diagnostic: mean confidence on wrong predictions is 0.49 and its ECE is the worst measured, so it is confidently wrong rather than underconfident. GCE’s bounded loss $L_q = (1 - p_y^q)/q$ produces smaller gradients as confidence rises, and lr 0.8 was tuned

for cross-entropy’s landscape. **Both results say “needs its own learning-rate search,” not “wrong for this dataset.”**

The one close call, and why a point estimate was not enough. At 15 epochs both augmentation policies’ training loss was still well above the control’s (3.32-3.43 vs. 3.02) – a signal that the proxy might simply be too short for augmentation to pay off, which none of the other axes showed as clearly. RandAugment therefore got a matched-epoch rematch: both legs re-run fresh for 30 epochs, since a cosine schedule needs its total length fixed upfront and cannot be resumed into. The gap narrowed from about 3 points to **0.84** (61.27% vs. 60.43%), with the control still ahead on macro-F1, weighted-F1 and worst-30. On point estimates alone that reads as a clean win. It is not: McNemar’s test on the shared images finds 454 the control alone got right against 401 for RandAugment, $n = 855$ discordant, $p = 0.075$ – **not significant**. The honest reading is that this proxy lacks the power to separate “the plain recipe is slightly better” from “these are tied.” The plain recipe carries forward on simplicity and an unbeaten record, not on a demonstrated margin over RandAugment.

Two non-levers. Batch size is flat: 160, 256 and 512 with the learning rate scaled linearly land at 55.85%, 55.67% and 55.30%, all tied against the control ($p = 0.74$, $p = 0.49$), and throughput is flat across the same range (Section 6), so batch size moves neither thing it could have. Raising the RandomResizedCrop scale floor above its ImageNet-inherited 0.08 – plausible, since that floor was tuned for 1.28M images against this set’s 90k – half-works instructively: the train/validation gap closes exactly as predicted (+11.05 to +4.31 to −0.56), but accuracy does not follow, tying at 0.25 and losing significantly at 0.40 ($p = 0.023$). Low-scale cropping is doing double duty as regularizer **and** view-diversity source; relaxing it makes the task easier to fit while removing diversity, a net loss at fixed epochs (Section C).

Why all six lost. The crop-scale result names the cause the other five share. At every schedule length tested, validation accuracy runs **above** training accuracy – +11.05 points at 15 epochs, +7.3 at 30. **This model underfits; it does not overfit.** Regularizing a model that has not finished fitting can only slow it down, which is exactly the ordering observed: the stronger the regularizer, the worse the result (Mixup below CutMix below RandAugment below plain). The one axis that did pay, learning rate, helps a model fit faster rather than generalize better. What it needed was schedule length, and Section 5.3 shows the gap closing to zero at 90 epochs.

Carried forward: the plain recipe – label-smoothed cross-entropy, no augmentation beyond crop-and-flip, no Mixup/CutMix, no EMA – at lr 0.8, batch 256, crop-scale floor 0.08.

5.3 Full supervised training run

One 90-epoch run was budgeted, and it was spent on exactly the configuration the sections above settled: the baseline [2, 2, 2, 1] 64-512 trunk, GAP head, plain recipe, lr 0.8, batch 256, crop-scale floor 0.08, nothing tuned for this run. Evaluation applies FixRes-style test-time resolution correction [13] – train at 176px, evaluate at Resize(256) + CenterCrop(224) – which main.py already does by default.

Table 4: The full 90-epoch run. val - test is read once, here.

	top-1	top-3	top-5
val - dev, best (epoch 86)	64.36%	82.22%	87.51%
val - dev, final (epoch 90)	64.26%	82.14%	87.61%
val - test (headline)	63.83%	81.79%	87.39%

val - test lands 0.53 points below val - dev’s best – the expected direction, since val - dev selected the checkpoint, and small enough to say the split is not doing anything surprising. Best and final differ by 0.1 points, against roughly 0.5 in the earlier validation run: no late overfitting.

What the recipe search bought. An earlier 90-epoch run on the same trunk at the pre-search default lr 0.1 reached 61.57% val - dev top-1; it was run to prove the pipeline survives full length, and it did, without crashes, NaNs or out-of-memory errors. The tuned run’s 64.36% is **+2.79 points**

276 **at identical trunk and epoch count**, almost all of it the learning rate, since that is the only one of
277 the six axes that moved.

278 **Schedule length is what was buying accuracy.** Read across the three schedule lengths at the same
279 recipe – 55.6% at 15 epochs, 61.3% at 30, 64.4% at 90 (val-dev) – and the returns diminish but never
280 stop. The training signal says why: train top-1 finished at 64.88% against 64.26% val, so the **+11.05-**
281 **point** validation-over-training gap measured at 15 epochs, and +7.3 at 30, has closed to roughly zero
282 only by epoch 90. The model spent every one of those epochs still fitting. This is the same diagnosis
283 Section 5.2 arrives at from the other direction, and it is why regularization had nothing to offer here.

284 Wall-clock was 2h05m against the earlier run’s 2h31m for identical work, with no slowdown over
285 the run (Section 6), and peak memory 1.99 GiB – under a quarter of what was available.

286 5.4 Self-supervised comparison

287 This comparison is implemented but **not run**; no numbers are claimed for it. SimSiam [12] is chosen
288 over BYOL [11] on memory and tuning grounds rather than reported accuracy: no momentum
289 encoder, so no second 6.5M-parameter shadow trunk in VRAM alongside the two augmented views
290 an epoch already costs, and no EMA-decay schedule to tune – and Section 5.2 is a cautionary result
291 about adding untuned hyperparameters to this budget. SimCLR [10] is excluded outright, degrading
292 below roughly 1,000-example batches that 8GB cannot reach here. FoodCNN.forward_features
293 exposes the pooled 512-d trunk output for a projector, at no cost to the parameter budget, and main.py
294 --init-encoder loads a pretrained trunk into a fresh classifier to finetune.

295 **The equal-compute control is not optional.** Pretraining (200 epochs, about 8 GPU-hours at two
296 views per step) plus finetuning (1.8) must be compared against spending those same hours on longer
297 supervised training of the same trunk, or the result cannot separate “pretraining helped” from “this
298 trunk keeps improving with more epochs regardless of objective.” Section 5.3 has already shown it
299 still gaining at epoch 90, which makes that alternative the likely one rather than a pedantic caveat.

300 6 Discussion

301 **Hardware measurement mattered as much as the architecture.** Several findings above exist
302 only because throughput and memory were measured rather than assumed: the tensor-core alignment
303 and spatial-resolution effects of Section 5.1; the dataloader never being the bottleneck at this
304 resolution, where crop-and-flip sustains 4.4-4.7k img/s against 1.7-1.9k of GPU demand and even
305 TrivialAugment keeps 2.5x headroom at eight workers, falling to 1.4x at sixteen as oversubscription
306 costs throughput on 15GB of system RAM; and batch size being throughput-flat from 160 to 768
307 while memory scales linearly to 5.64 GiB, meaning the GPU is compute-saturated at the smallest
308 batch tried, not idle.

309 **A power cap, not a thermal wall, and a short benchmark cannot tell you which.** The GPU runs
310 against a roughly 50W cap with its clock oscillating 2175-2340 MHz against a 3090 MHz maximum,
311 and an early version of the throughput benchmark swung 45% on one variant between sweeps before
312 being changed to time three 40-step rounds and keep the best (sweeps now agree within 2-5%, and
313 rankings are stable). The 90-epoch pipeline-validation run then came in 28% over its own synthetic
314 estimate, with per-batch time degrading from 0.16s to 0.26s past epoch 60 – which looks exactly like
315 heat-soak that no short benchmark window reproduces. It was not: the same trunk, at the same length,
316 held 0.147-0.169s/step throughout after the machine was switched to its performance power profile
317 (Section 5.3). The two runs are not a controlled comparison, but the reading changes the conclusion
318 one would draw from either alone. Throughput figures here remain upper bounds and ratios between
319 variants rather than wall-clock promises.

320 **Limitations.** **No run was seeded** (Section 3), so every figure is a single point estimate. This binds
321 hardest on the two closest comparisons – the 1.2-point spread across three candidates in Section 5.1,
322 and RandAugment’s 0.84-point matched-epoch deficit that McNemar’s test already declines to call
323 significant – and a second seed on those legs is the cheapest available strengthening of this report.
324 Separately, every recipe axis was evaluated at the learning rate tuned for the plain recipe, so GCE
325 and EMA plausibly lost to that choice rather than on their merits: unproven here, not ruled out. All

measurements are specific to one machine and need not transfer even in ratio, and training is from scratch throughout, so these accuracies are not comparable to pretrained baselines.

7 Conclusion

A plain residual CNN with a genuinely global pooling head reaches **63.83% top-1 / 87.39% top-5** on FoodX-251 from scratch, on a test split read once, using 66% of the parameters it was allowed. Both findings behind that number cut against where effort naturally goes: capacity never constrained this model, and neither did overfitting – what it wanted was learning rate and time, and the budget spent ruling out six regularizers is what established that. Throughput and memory acted as design constraints alongside the parameter cap throughout. Two things would most improve this report: reseeding the two comparisons too close for one run to settle, and running Section 5.4 against its equal-compute control.

References

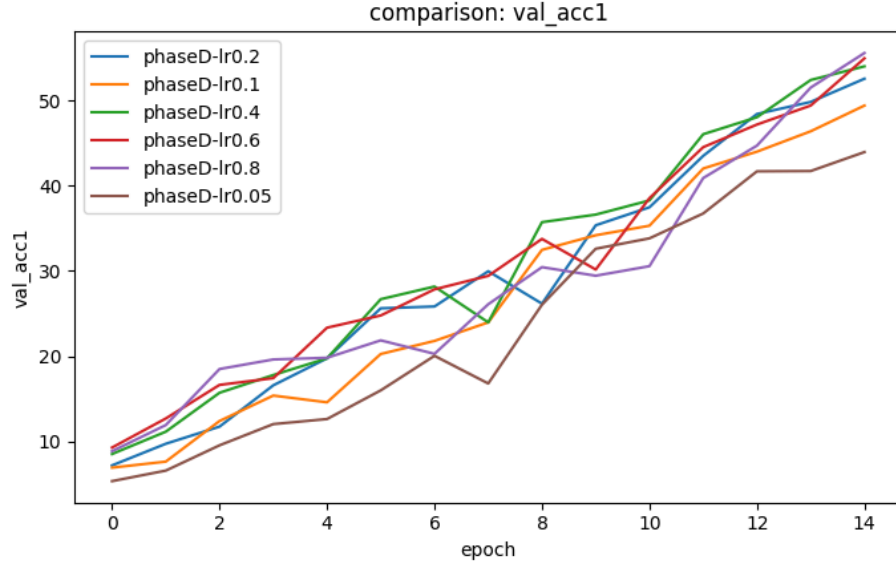
- [1] P. Kaur, K. Sikka, W. Wang, S. Belongie, and A. Divakaran, “FoodX-251: A Dataset for Fine-grained Food Classification,” *arXiv preprint arXiv:1907.06167*, 2019.
- [2] D. Qin *et al.*, “MobileNetV4 - Universal Models for the Mobile Ecosystem,” *arXiv preprint arXiv:2404.10518*, 2024.
- [3] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, “MobileOne: An Improved One millisecond Mobile Backbone,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 7907–7917.
- [4] S. Woo *et al.*, “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 16133–16142.
- [5] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, “RandAugment: Practical automated data augmentation with a reduced search space,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2020, pp. 702–703.
- [6] S. G. Müller and F. Hutter, “TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.
- [7] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,” in *International Conference on Learning Representations*, 2018.
- [8] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, “CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019.
- [9] Z. Zhang and M. Sabuncu, “Generalized Cross Entropy Loss for Training Deep Neural Networks with Noisy Labels,” in *Advances in Neural Information Processing Systems*, 2018.
- [10] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 1597–1607.
- [11] J.-B. Grill *et al.*, “Bootstrap Your Own Latent - A New Approach to Self-Supervised Learning,” in *Advances in Neural Information Processing Systems*, 2020, pp. 21271–21284.
- [12] X. Chen and K. He, “Exploring Simple Siamese Representation Learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.
- [13] H. Touvron, A. Vedaldi, M. Douze, and H. Jégou, “Fixing the train-test resolution discrepancy,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.

A Technical appendices and supplementary material

Supplementary material referenced from Section 4, Section 5.1 and Section 5.2 follows: the learning-rate sweep in full, the batch-size and crop-scale tables, the synthetic throughput sweep behind the architecture search, and additional detail on the rejected depthwise-separable trunk.

373 B Learning-rate sweep

374 Section 5.2 reports the learning-rate sweep as final accuracies. The per-epoch curves behind those
 375 numbers, all 15-epoch val-dev proxies on the baseline trunk at batch 256:



376 Figure 1: val-dev top-1 per epoch across the learning-rate sweep. The visible mid-run dip on lr 0.8
 377 around epochs 8-9, absent from 0.4 and 0.6, is the growing-noise half of the argument for stopping
 378 the search there; its training loss (not shown) is the smoothest of the sweep and reaches the lowest
 379 final value.

380 Table 5: Learning-rate sweep, 15-epoch val-dev proxies, baseline trunk, batch
 381 256. Top-3 and top-5 were only logged from the lr 0.4 leg onward, when
 382 top-3 tracking was added to RunLog mid-sweep.

383	LR	top-1	top-3	top-5	Δ vs. previous
384	0.05	43.96%	—	—	—
385	0.10	49.41%	—	—	+5.45
386	0.20	52.56%	—	—	+3.15
387	0.40	54.02%	74.16%	81.02%	+1.46
388	0.60	54.96%	—	—	+0.94
389	0.80	55.58%	74.75%	81.40%	+0.62

390 lr 0.10 is the reference script’s default, and the setting the pipeline validation run in Section 5.3 used;
 391 lr 0.80 is what Section 5.2 carries forward.

392 C Batch size and crop scale

393 Both axes are summarized in Section 5.2; the full tables follow. Each row is a 15-epoch val-dev
 394 proxy on the baseline trunk with the plain recipe, compared against the lr 0.8 / batch 256 control by
 395 a paired McNemar exact test.

396 Table 6: Batch size with the learning rate scaled linearly from the batch-256 control. All three are
 397 statistically tied.

398	Batch	LR	top-1	Δ	McNemar p	Peak VRAM
399	160	0.5	55.85%	+0.18	0.7407	1.30 GiB
400	256	0.8	55.67%	—	—	1.99 GiB
401	512	1.6	55.30%	−0.36	0.4868	3.83 GiB

402 One infrastructure finding came out of this leg: batch 512 at the default eight dataloader workers
 403 crashed one epoch in with `RuntimeError: unable to`
 404 `allocate shared memory`, and re-ran clean at four workers. Each worker prefetches collated batches,
 405 and at batch 512 that overruns this machine’s 7.7 GiB /dev/shm well before it overruns the 8 GiB
 406 of VRAM. **At large batch sizes on this box, shared memory binds before GPU memory does** –
 407 worth knowing before reading a batch-size ceiling off VRAM figures alone.

408 Table 7: RandomResizedCrop scale floor. The train/validation gap behaves exactly as the underfitting
 409 hypothesis predicts; accuracy does not follow it.

410	Scale min	top-1	Δ	McNemar p	train top-1	gap
411	0.08	55.67%	—	—	44.53%	+11.05
412	0.25	55.71%	+0.05	0.9465	51.36%	+4.31
413	0.40	54.49%	−1.17	0.0232	55.19%	−0.56

414 No matched-epoch rematch was run for this axis, unlike RandAugment’s. The reason is visible in the
 415 table: the 0.40 leg already trains **faster** per epoch than the control (55.19% vs. 44.53% train top-1 at
 416 epoch 15) and still loses on validation, so there is no “not converged yet” argument for more epochs
 417 to close the gap.

418 The control row’s top-1 here is the checkpoint re-score used for every McNemar comparison in this
 419 report, rather than the epoch-end log value quoted in Section 5.2 – the same run read two ways.
 420 Checkpoint re-scores run in bf16 and read about 0.03 points above the fp32 per-epoch logs; every
 421 comparison pairs like with like.

422 D Full architecture-screening sweep

423 Table 1 in the main text reports 15-epoch accuracy proxies for four trunk variants. Those four
 424 were selected from a larger synthetic screening pass over throughput, parameter count, and memory
 425 alone (`benchmarks/bench.py`, `benchmarks/trunk_variants.py`), all at 176px training resolution
 426 and batch 160, before any accuracy was measured. The full sweep:

Table 8: Synthetic throughput/memory sweep over trunk variants under the 10M-parameter cap, 176px training resolution, batch 160. “90 ep” is the projected wall-clock for a full 90-epoch run at the measured throughput; “Final map” is the trunk’s spatial resolution immediately before global average pooling.

Variant	Params	img/s	90 ep	Peak VRAM	Final map
baseline [2, 2, 2, 1] 64-512	6.58M	1688	1.8 h	1.26 GiB	6x6
[2, 2, 3, 1] 64-512	7.76M	1585	1.9 h	1.31 GiB	6x6
[2, 2, 4, 1] 64-512	8.94M	1475	2.0 h	1.36 GiB	6x6
[2, 3, 3, 1] 64-512	8.06M	1460	2.0 h	1.39 GiB	6x6
[2, 2, 2, 2] 64-448	9.46M	1608	1.8 h	1.30 GiB	6x6
wider@11 64-320-512	7.98M	1494	2.0 h	1.30 GiB	6x6
5 stages 48-512 [2, 2, 2, 2, 1]	9.08M	1960	1.5 h	1.04 GiB	3x3
uniformly wider 72-576	8.31M	1062	2.8 h	1.41 GiB	6x6
5 stages, no stem max-pool	9.08M	684	4.3 h	2.16 GiB	6x6

The last row isolates the resolution effect cited in Section 6: identical parameter count to the row above it, but with the stem’s max-pool removed to keep the final feature map at 6x6 instead of 3x3, at a cost of 2.9x throughput and roughly 2x peak memory. The “uniformly wider 72-576” row isolates the tensor-core alignment effect: fewer parameters than the five-stage variant, at 46% lower throughput, purely from using channel widths that are not multiples of 32.

E The rejected depthwise-separable trunk

Before the residual trunk in Section 4, a MobileNetV2-style depthwise-separable version of the same network was implemented and measured head-to-head on the same hardware. It is kept in `src/model.py`, unused by `FoodCNN.features`, as a documented negative result rather than deleted:

Table 9: Depthwise-separable vs. plain-convolution trunk, measured head-to-head on the project GPU (176px, batch 160, bf16, `channels_last`).

Trunk	Params	img/s	Peak VRAM
Depthwise-separable (MobileNetV2-style)	2.55M	621	3.83 GiB
Plain-convolution residual (adopted)	6.58M	918	1.28 GiB

Fewer parameters did not mean faster or lighter here: depthwise convolutions trade parameter count for activation memory and wall-clock, and under this project’s binding constraints – throughput and memory, not raw parameter count – that trade loses.¹ The plain-convolution trunk was adopted on this basis before the architecture search in Section 5.1 ever began.

¹This comparison’s 918 img/s baseline figure is lower than the 1688 img/s recorded for the same trunk in Table 8. The two were measured in separate benchmarking passes – this comparison predates `benchmarks/bench.py`’s current three-round, keep-the-best methodology – and, per the thermal-throttling behavior discussed in Section 6, absolute throughput on this hardware is not stable across passes. The qualitative conclusion, that the depthwise-separable trunk trades parameters for memory and wall-clock, does not depend on which figure is used.

459 NeurIPS Paper Checklist

460 1. Claims

461 Question: Do the main claims made in the abstract and introduction accurately reflect the paper's
462 contributions and scope?

463 Answer: [Yes]

464 Justification: The abstract and Section 1 state the headline 63.83% top-1 / 87.39% top-5 on val -
465 test, the parameter budget actually used (6.58M of 10M), and the two findings the paper rests
466 on – that capacity does not bind (Table 2) and that the model underfits, so no regularizer helped
467 (Table 3). The self-supervised comparison named in the title is explicitly marked as designed
468 but unrun, in the abstract, Section 1 and Section 5.4 alike, and no numbers are claimed for it.

469 Guidelines:

- 470 • The answer NA means that the abstract and introduction do not include the claims made in
471 the paper.
- 472 • The abstract and/or introduction should clearly state the claims made, including the contri-
473 butions made in the paper and important assumptions and limitations. A No or NA answer
474 to this question will not be perceived well by the reviewers.
- 475 • The claims made should match theoretical and experimental results, and reflect how much
476 the results can be expected to generalize to other settings.
- 477 • It is fine to include aspirational goals as motivation as long as it is clear that these goals are
478 not attained by the paper.

479 2. Limitations

480 Question: Does the paper discuss the limitations of the work performed by the authors?

481 Answer: [Yes]

482 Justification: Section 6 has a dedicated Limitations paragraph. It states that no run was seeded,
483 which comparisons that most affects, that all measurements come from one machine and need not
484 transfer, that from-scratch accuracies are not comparable to pretrained baselines, and that GCE
485 and EMA were each evaluated at a learning rate tuned for a different loss and so are unproven
486 rather than ruled out.

487 Guidelines:

- 488 • The answer NA means that the paper has no limitation while the answer No means that the
489 paper has limitations, but those are not discussed in the paper.
- 490 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 491 • The paper should point out any strong assumptions and how robust the results are to
492 violations of these assumptions (e.g., independence assumptions, noiseless settings, model
493 well-specification, asymptotic approximations only holding locally). The authors should
494 reflect on how these assumptions might be violated in practice and what the implications
495 would be.
- 496 • The authors should reflect on the scope of the claims made, e.g., if the approach was only
497 tested on a few datasets or with a few runs. In general, empirical results often depend on
498 implicit assumptions, which should be articulated.
- 499 • The authors should reflect on the factors that influence the performance of the approach. For
500 example, a facial recognition algorithm may perform poorly when image resolution is low
501 or images are taken in low lighting. Or a speech-to-text system might not be used reliably
502 to provide closed captions for online lectures because it fails to handle technical jargon.
- 503 • The authors should discuss the computational efficiency of the proposed algorithms and
504 how they scale with dataset size.

- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory Assumptions and Proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: The paper contains no theoretical results.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental Result Reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4 gives the architecture in full, Section 5.2 and Section B give every hyperparameter and how it was chosen, and Section 3 specifies the split protocol, including that `src/make_val_split.py` regenerates a byte-identical `val-dev/val-test` split from the official labels and a fixed seed (251). The exact command behind the headline run is recorded with the run log. One caveat is stated rather than hidden: training runs were not seeded (Section 3), so a rerun reproduces the setup and the conclusions but not the exact digits.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for

553 how to replicate the results, access to a hosted model (e.g., in the case of a large language
 554 model), releasing of a model checkpoint, or other means that are appropriate to the research
 555 performed.

- 556 • While NeurIPS does not require releasing code, the conference does require all submissions
 557 to provide some reasonable avenue for reproducibility, which may depend on the nature of
 558 the contribution. For example

- 559 (a) If the contribution is primarily a new algorithm, the paper should make it clear how to
 560 reproduce that algorithm.
- 561 (b) If the contribution is primarily a new model architecture, the paper should describe the
 562 architecture clearly and fully.
- 563 (c) If the contribution is a new model (e.g., a large language model), then there should either
 564 be a way to access this model for reproducing the results or a way to reproduce the model
 565 (e.g., with an open-source dataset or instructions for how to construct the dataset).
- 566 (d) We recognize that reproducibility may be tricky in some cases, in which case authors
 567 are welcome to describe the particular way they provide for reproducibility. In the case
 568 of closed-source models, it may be that access to the model is limited in some way (e.g.,
 569 to registered users), but it should be possible for other researchers to have some path to
 570 reproducing or verifying the results.

571 5. Open Access to Data and Code

572 Question: Does the paper provide open access to the data and code, with sufficient instructions
 573 to faithfully reproduce the main experimental results, as described in supplemental material?

574 Answer: [Yes]

575 Justification: FoodX-251 [1] is a public benchmark obtained from its official release; it is not
 576 redistributed here. All training, benchmarking and analysis code is in the accompanying repos-
 577 itory, and the validation split is regenerated by a script rather than shipped as data, so it cannot
 578 silently drift from the one used here.

579 Guidelines:

- 580 • The answer NA means that paper does not include experiments requiring code.
- 581 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/
 582 public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 583 • While we encourage the release of code and data, we understand that this might not be
 584 possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including
 585 code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- 586 • The instructions should contain the exact command and environment needed to run to
 587 reproduce the results. See the NeurIPS code and data submission guidelines ([https://
 588 neurips.cc/public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 589 • The authors should provide instructions on data access and preparation, including how to
 590 access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 591 • The authors should provide scripts to reproduce all experimental results for the new pro-
 592 posed method and baselines. If only a subset of experiments are reproducible, they should
 593 state which ones are omitted from the script and why.
- 594 • At submission time, to preserve anonymity, the authors should release anonymized versions
 595 (if applicable).
- 596 • Providing as much information as possible in supplemental material (appended to the paper)
 597 is recommended, but including URLs to data and code is permitted.

598 6. Experimental Setting/Details

599 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpara-
 600 meters, how they were chosen, type of optimizer) necessary to understand the results?

601 Answer: [Yes]

602 Justification: Section 4 gives the optimizer (SGD with Nesterov momentum), schedule (cosine
603 with five-epoch linear warmup), loss, precision policy, crop sizes and worker count; Section 5.2
604 gives the learning rate and batch size with the sweeps that chose them; Section 3 gives the splits
605 and the 15-epoch proxy protocol. Section B and Section C give the sweeps in full.

606 Guidelines:

- 607 • The answer NA means that the paper does not include experiments.
- 608 • The experimental setting should be presented in the core of the paper to a level of detail
609 that is necessary to appreciate the results and make sense of them.
- 610 • The full details can be provided either with the code, in appendix, or as supplemental
611 material.

612 7. Experiment Statistical Significance

613 Question: Does the paper report error bars suitably and correctly defined or other appropriate
614 information about the statistical significance of the experiments?

615 Answer: [Yes]

616 Justification: Every close comparison is accompanied by a paired McNemar exact test on the
617 same val-dev images (benchmarks/significance_test.py), reported as an exact p in Table 2,
618 Table 3, Table 6 and Table 7. That test is what keeps two results honest: the 64-384 trunk’s tie
619 with the baseline ($p = 1.0$) and RandAugment’s 0.84-point deficit, which is **not** significant ($p =$
620 0.075) and is reported as unproven rather than a win. We do **not** report error bars over random
621 seeds, because no run was seeded; Section 3 and Section 6 both state this and name it as the first
622 thing that should be added, and are explicit that McNemar covers sampling variance over images
623 only, not training variance.

624 Guidelines:

- 625 • The answer NA means that the paper does not include experiments.
- 626 • The authors should answer “Yes” if the results are accompanied by error bars, confidence
627 intervals, or statistical significance tests, at least for the experiments that support the main
628 claims of the paper.
- 629 • The factors of variability that the error bars are capturing should be clearly stated (for
630 example, train/test split, initialization, random drawing of some parameter, or overall run
631 with given experimental conditions).
- 632 • The method for calculating the error bars should be explained (closed form formula, call to
633 a library function, bootstrap, etc.)
- 634 • The assumptions made should be given (e.g., Normally distributed errors).
- 635 • It should be clear whether the error bar is the standard deviation or the standard error of
636 the mean.
- 637 • It is OK to report 1-sigma error bars, but one should state it. The authors should preferably
638 report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality
639 of errors is not verified.
- 640 • For asymmetric distributions, the authors should be careful not to show in tables or figures
641 symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- 642 • If error bars are reported in tables or plots, the authors should explain in the text how they
643 were calculated and reference the corresponding figures or tables in the text.

644 8. Experiments Compute Resources

645 Question: For each experiment, does the paper provide sufficient information on the computer
646 resources (type of compute workers, memory, time of execution) needed to reproduce the
647 experiments?

648 Answer: [Yes]

Justification: All runs used one RTX 5050 Laptop GPU (8GB VRAM) in its performance power profile, stated in Section 5. Per-run wall-clock and peak VRAM are logged for every run (`src/runlog.py`) and quoted in the text: about 18-27 minutes per 15-epoch proxy, 2h05m and 1.99 GiB peak for the 90-epoch headline run. The runs behind the reported results total roughly 7 GPU-hours. The project spent appreciably more than that, and the paper says so rather than only counting what worked: the synthetic screening sweep of Section D, a separate 90-epoch pipeline-validation run (Section 5.3), the two 30-epoch legs of the matched rematch, and one proxy leg lost to a shared-memory crash and re-run (Section C) are all additional.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of Ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work uses a public benchmark under its terms, releases its code, involves no human subjects or personally identifying data, and discloses its negative results and its methodological gaps rather than reporting only what worked.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader Impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [NA]

Justification: The work trains a small image classifier on a public food benchmark, entirely from scratch, and releases no model or data that carries foreseeable societal risk beyond that of image classification generally.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any

negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: No data or model with a high risk of misuse is released. The trained classifier predicts one of 251 food categories and has no generative capability.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for Existing Assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: FoodX-251 is credited to [1] and used under its published terms. `src/main.py` is adapted from the PyTorch ImageNet reference training script and says so in the repository, keeping that script's structure deliberately; `src/resnet.py` is kept unmodified as a baseline for comparison. PyTorch and torchvision are used as ordinary dependencies under their own licenses. Every technique tested in Section 5.2 is cited to its originating paper.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, <https://paperswithcode.com/datasets> has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New Assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [NA]

Justification: No new dataset or pretrained model is released. The accompanying code is documented in the repository, but it is an implementation of the paper rather than an asset offered for reuse.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and Research with Human Subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: The paper involves no crowdsourcing and no human subjects. FoodX-251's validation labels were human-verified by its original authors, not by us.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

789 Justification: The paper involves no research with human subjects.

790 Guidelines:

- 791 • The answer NA means that the paper does not involve crowdsourcing nor research with
- 792 human subjects.
- 793 • Depending on the country in which research is conducted, IRB approval (or equivalent)
- 794 may be required for any human subjects research. If you obtained IRB approval, you should
- 795 clearly state this in the paper.
- 796 • We recognize that the procedures for this may vary significantly between institutions and
- 797 locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines
- 798 for their institution.
- 799 • For initial submissions, do not include any information that would break anonymity (if
- 800 applicable), such as the institution conducting the review.

801 **16. Declaration of LLM Usage**

802 Question: Does the paper describe the usage of LLMs if it is an important, original, or non-

803 standard component of the core methods in this research? Note that if the LLM is used only for

804 writing, editing, or formatting purposes and does *not* impact the core methodology, scientific

805 rigor, or originality of the research, declaration is not required.

806 Answer: [NA]

807 Justification: No LLM is a component of the method. The classifier, the training pipeline and

808 every experiment reported here are ordinary supervised computer vision, and no result depends

809 on an LLM. LLM assistance was used for writing and for routine software engineering, which

810 the policy above does not require declaring.

811 Guidelines:

- 812 • The answer NA means that the core method development in this research does not involve
- 813 LLMs as any important, original, or non-standard components.
- 814 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not be
- 815 described.